Liu *et al. International Journal of STEM Education* (2026) 13:10
https://doi.org/10.1186/s40594-025-00592-w

International Journal of STEM Education

RESEARCH **Open Access**

Tool, tutor, or crutch?: A grounded theory of cognitive scaffolding and offloading in AI-assisted programming education

Dandan Liu[1,2*], Guangrui Fan[1*] and Lihu Pan[1*]

Abstract

Motivation The integration of generative AI into programming education has produced a widely reported tension between performance and learning. We distinguish immediate task performance from genuine learning: durable, transferable conceptual understanding and evaluative skill, and examine how AI support shapes learning processes, not merely outcomes. While many studies document improved speed, accuracy, and affect with AI support, questions remain about the quality of underlying learning.

Method and participants We use a constructivist grounded-theory design with constant comparison across two naturally occurring course sections (an AI-enabled section and a human pair-programming section used as a theoretical contrast). Over one semester, we collected triangulation across data sources from undergraduate Java programming students–interaction logs, pre-/post-course concept maps, and semi-structured dyadic interviews (AI-enabled: $N = 24$; theoretical contrast: $N = 17$).

Findings Analysis revealed a core tension between students' pursuit of "Domain Mastery" (conceptualization, explanation, and evaluation) and "Tool Mastery" (procedural efficiency with AI). We identified dynamic strategy switching (the Strategic Dance), Partnership Framing with an Illusion of Dialogue subtheme, and two recurrent evaluation challenges (Trust-but-Can't-Verify for novices; a Boilerplate Blindspot for more experienced students). We also describe attenuated meta-cognitive calibration–a mismatch between perceived readiness and independent capability–co-occurring with sustained offloading patterns. These categories synthesize into a process-level tension model with two recurrent loops (Scaffolding and Offloading), interpreted through Cognitive Load Theory and Self-Determination Theory.

Conclusion We offer a theory-building account that helps explain how widely observed performance and affect gains can co-occur with thinner opportunities for germane processing and authorship. The model generates testable implications (e.g., critique-the-AI phases, planned fading, verification journals), and we invite multi-site tests to evaluate boundary conditions.

*Correspondence:
Dandan Liu
S2134717@siswa.um.edu.my
Guangrui Fan
fgr@tyust.edu.cn
Lihu Pan
panlh@tyust.edu.cn

Full list of author information is available at the end of the article

© The Author(s) 2026. **Open Access** This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

Keywords Generative AI, Programming education, Cognitive scaffolding, Cognitive offloading, Grounded theory, Metacognition, Self-efficacy

Introduction

The rapid and widespread integration of generative artificial intelligence (AI) represents a paradigmatic shift in the landscape of higher education. Emerging technologies, including generative AI, are reshaping STEM education by enabling new forms of interaction, personalization, and accessibility (Chiu & Li, 2023; Li, 2025). Tools such as GitHub Copilot, ChatGPT, and Claude now autocomplete functions, supply code explanations, and engage learners in natural-language dialogue across almost every major university programming curriculum (Denny et al., 2024). Early adoption has been rapid: surveys suggest that more than 70% of recent graduates expect generative AI (GenAI) support to be formally integrated into their courses (Wu et al., 2025a). These technologies promise to democratize access to high-quality assistance, offering "on-demand pair programming" at a scale that human teaching staff cannot match. In this paper, we distinguish immediate task performance from genuine learning, the latter understood as durable, transferable conceptual understanding and evaluative skill. This distinction frames how we interpret the role of AI support in programming education.

Empirical evidence suggests that AI assistance can improve short-term performance. In a quasi-experimental study with 234 undergraduates, Fan et al. (2025) reported that an AI pair-programming partner significantly improved task completion accuracy and reduced coding anxiety relative to both solo work and human pair programming. Similar gains have been documented in controlled lab experiments (Yilmaz & Yilmaz, 2023) and semester-long field studies (Lyu et al., 2024). In some studies of complex maintenance tasks, GitHub Copilot users reported or demonstrated faster completion times while writing less boilerplate and conducting fewer external searches (Shihab et al., 2025). These findings mirror professional-developer reports of double-digit productivity gains. Two recent meta-analyses corroborate these single-study findings. The results confirm that GenAI has positive effects on a range of student outcomes, including lower- and higher-order cognitive skills, non-cognitive skills, motivation, and engagement. The analyses also identify key moderating factors, including discipline, learning strategy, and usage context (Xia et al., 2025; Qi et al., 2025). Such results indicate that AI can meaningfully enhance performance and engagement, yet performance gains alone do not establish that deep, transferable learning has occurred.

Yet these performance improvements raise a critical question: do they reflect genuine learning facilitated by effective scaffolding, or do they co-occur with superficial dependency enabled by cognitive offloading? (Yan et al., 2025). Scaffolding, in a Vygotskian sense, is temporary, adaptive support that helps a learner master a task just beyond their independent ability. In contrast, cognitive offloading occurs when a learner delegates essential mental work to an external tool, bypassing the productive struggle (Leinwand et al., 2014; Kapur, 2008) necessary to build robust, long-term knowledge schemas (Jose et al., 2025). This concern is empirically noted as well. Studies now warn that students who rely heavily on GenAI often struggle to articulate why autogenerated code works (Jose et al., 2025) and score lower on delayed conceptual tests (Bastani et al., 2024). Observational data further reveal reductions in independent debugging and problem decomposition–the very practices thought to cultivate durable expertise (Shihab et al., 2025). This tension between visible success and invisible understanding has been labelled the performance–learning paradox (Yan et al., 2025).

While the predominantly quantitative literature has documented the prevalence and correlates of this paradox, it leaves a methodological blind spot regarding the moment-to-moment processes by which students use AI. Outcome-focused studies seldom unpack the rich, moment-to-moment process of how students interact with AI, why they choose certain strategies, or how their metacognitive beliefs evolve over time. Recent meta-analytic syntheses further underscore heterogeneity and moderators, strengthening the case for process-level inquiry that can explain how different patterns of use relate to learning mechanisms (Xia et al., 2025; Qi et al., 2025). We therefore foreground three gaps: (i) limited process-tracing of student–AI interaction (how strategies form and shift), (ii) limited attention to how students evaluate, verify, and integrate AI outputs, and (iii) little evidence on metacognitive calibration over time (how perceived competence aligns with independent capability). To move from identifying the paradox to explaining it, we must look beyond whether AI helps performance and ask how and under what conditions it shapes learning. Accordingly, we adopt a constructivist grounded-theory design across two naturally occurring instructional ecologies (AI-enabled and human pair programming) used as a theoretical contrast, not a control; we refrain from causal claims and use the contrast to sharpen categories. We explore the following questions:

- RQ1 (Strategies): What specific interaction strategies do students develop when using an AI assistant for

programming tasks, and how can these strategies be categorized in relation to learning objectives?
- RQ2 (Evaluation): How do students critically evaluate, verify, and integrate AI-generated code and explanations into their own work, and what factors influence the depth of this critical engagement?
- RQ3 (Metacognition & Self-Efficacy): How does sustained use of an AI programming assistant influence metacognitive calibration, self-efficacy, awareness of knowledge gaps, and perceived ability to solve programming problems independently?

By addressing these questions, this study makes a process-level, theory-building contribution to the field. Analyzing student strategies (RQ1) and evaluative practices (RQ2) provides a process-level view that existing quantitative work lacks. Tracking metacognitive calibration and self-efficacy (RQ3) uncovers potential longer-term risks and benefits associated with AI-supported workflows. The model developed in this study theorizes the dynamic tension between Domain Mastery and Tool Mastery. We position this model as an interpretive account that identifies conditions under which AI functions as a scaffold versus a shortcut, offering practical implications for instructors, curriculum designers, and tool builders.

Theoretical frameworks and literature review

Guiding frameworks

We center our analysis on two complementary lenses: Cognitive Load Theory (CLT) for cognitive mechanisms and Self-Determination Theory (SDT) for motivational dynamics. These provide the primary constructs through which we interpret observed interaction patterns (e.g., shifts between inquiry and delegation; verification practices; metacognitive calibration). In addition, we use Vygotsky's Zone of Proximal Development (ZPD) and Cognitive Apprenticeship as sensitizing concepts–heuristics that shape what we attend to–rather than as frameworks we formally integrate or test. CLT and SDT are commensurable in this context because they address different but compatible levels of analysis: CLT describes how instructional supports redistribute cognitive effort across extraneous and germane processes, while SDT describes how supports and norms shape perceived competence, autonomy, and relatedness. We therefore treat performance patterns as potentially arising from changes in cognitive load allocation and from shifts in motivational appraisals. ZPD and Cognitive Apprenticeship inform our attention to scaffolding and fading but are not modeled causally.

AI as a partner in programming education

Research on automated support for programming has progressed through three major waves. First-generation intelligent tutoring systems (ITS) of the 1980 s–2000 s, such as the Lisp Tutor, relied on expert-model rules and step-level feedback; meta-analyses consistently showed medium-to-large learning gains but highlighted high authoring costs and limited domain coverage (Létourneau et al., 2025). Second-generation adaptive systems in the 2010 s introduced data-driven learner models (e.g., Bayesian knowledge tracing), reducing development overhead yet remaining confined to narrowly scripted problem types (Gupta & Bostrom, 2009).

Large language models (LLMs) represent a third wave. Trained on vast code and prose corpora, tools such as GitHub Copilot and ChatGPT offer open-ended dialogue, context-aware code generation, and on-demand explanation without handcrafted pedagogy (Denny et al., 2024). Across multiple empirical contexts–short laboratory tasks (Yilmaz & Yilmaz, 2023), semester-long field courses (Lyu et al., 2024), and brownfield maintenance challenges (Shihab et al., 2025)–LLM assistance has been associated with faster completion, higher surface accuracy, and improved affect. Nevertheless, researchers caution that these gains may mask a lack of deep conceptual growth and hinder long-term knowledge transfer (Jeyaraj & Sethi, 2025), prompting calls for theory-driven analyses of how AI changes cognition and motivation (Chiu et al., 2023; Fan et al., 2025; Jose et al., 2025). In this paper, we adopt the stance that AI functions as a powerful cultural tool whose pedagogical value depends on human mediation (instructional design, norms, and assessment), rather than as a replacement "tutor." Recent studies test designs that intentionally channel GenAI toward reflective scaffolding rather than uncritical delegation. Lee et al. (2025) generate *personalized reflective reports* from students' error codes; in a quasi-experimental study, this reported increases in programming self-efficacy and higher-order thinking while improving project quality–precisely the sort of germane processing our Scaffolding Loop predicts. In a randomized trial, Wu et al. (2025b) embed ChatGPT into a *peer-assessment cycle* (PA-GPT), treating the model as a "virtual peer." PA-GPT significantly outperformed conventional ChatGPT use on knowledge construction, higher-order thinking skills (HOTS), and STEM attitudes, addressing the social-verification gap we note in solo student–AI work and contrasting with our Illusion-of-Dialogue subtheme.

Theories and sensitizing concepts for AI-supported learning

Cognitive load theory (CLT): primary lens

CLT offers constructs to analyze how AI-supported workflows redistribute cognitive effort (Gkintoni et al., 2025). Total load is decomposed into intrinsic (task complexity), extraneous (avoidable inefficiencies), and germane (schema-building) components (Debue & Van De

Leemput, 2014). AI tools often reduce extraneous load (less boilerplate search, fewer syntax stalls) (Jose et al., 2025). Two risks follow: First, for high element interactivity tasks, direct solutions can obscure necessary coordination work, inviting premature acceptance and thus limiting germane processing (Stadler et al., 2024). Second, the expertise reversal effect implies that helpful supports for novices may become redundant for more experienced learners, adding extraneous overhead (Kalyuga, 2007). In our interpretation, scaffolding is evident when extraneous load is reduced while learners still engage in germane activities (modifying, testing, explaining AI output); offloading is evident when both extraneous and germane processing are minimized.

Self-determination theory (SDT): primary lens

SDT complements CLT by addressing how tools and norms influence motivation. Learning flourishes when needs for competence, autonomy, and relatedness are supported (Niemiec & Ryan, 2009; Xia et al., 2022). Prior work has shown that teacher support plays a critical role in sustaining student motivation when learning with AI-based tools (Chiu et al., 2024). AI-enabled completion can bolster perceived competence (fewer blocks, more progress) (Dung et al., 2024; Boguslawski et al., 2025). Yet depending on task design and classroom norms, frequent delegation may reduce authorship and evaluative control, creating a competence–autonomy tension (Khlaif et al., 2025): learners feel effective because the tool produces working code, while their sense of volitional control may decline if they do not own the reasoning. Over-reliance on impersonal tool interaction can also leave relatedness needs under-served relative to peer or instructor collaboration (Fan et al., 2025).

Table 1 Mapping of literature gaps to research questions

Limitation in prior research	Theoretical implication	Addressed by
Outcome-focused studies (e.g., grades, completion time) lack process-tracing of student-AI interaction	The how and why of student strategies–whether they use AI for scaffolding or offloading–are unknown (CLT: germane vs. extraneous engagement)	RQ1 (Strategies)
AI is often framed as an ideal tutor, overlooking the cognitive challenge of evaluating its potentially flawed or opaque output	The critical process of verification, central to Cognitive Apprenticeship (sensitizing) and germane load (CLT), is underexplored	RQ2 (Evaluation)
Focus on short-term performance gains masks the potential for long-term effects on independent problem-solving ability	The cumulative impact on metacognitive calibration (CLT: opportunities for germane processing) and the competence–autonomy tension (SDT) is unclear	RQ3 (Metacognition)

Vygotsky's ZPD (Sensitizing concept)

ZPD focuses attention on how temporary, adaptive support enables independence (Vygotsky, 1978; Shabani et al., 2010). We do not treat AI as a literal "More Knowledgeable Other." Instead, we use ZPD as a sensitizing concept to ask whether observed supports function as scaffolds (fading, adaptivity, internalization) or as shortcuts that displace learning. Consistent with sociocultural emphases on intersubjectivity (Stojanov, 2023; Medika, n.d.), we frame AI as a cultural tool mediated by human pedagogy rather than as a replacement MKO.

Cognitive apprenticeship (Sensitizing concept)

Cognitive Apprenticeship highlights modeling, coaching, and fading (Collins et al., 1991). We treat it as a sensitizing lens to interrogate whether supports exhibit fading and coaching characteristics. While current tools readily model solutions, they are often always-on by default; without intentional design, this can blur the line between scaffolding and dependency (Shah & Soosai Raj, 2024).

Research gaps

Individually, these perspectives illuminate cognitive (CLT), motivational (SDT), and instructional (ZPD/Apprenticeship) facets of AI-supported learning. Integrating these insights, we adopt an anchored synthesis: our primary explanations draw on CLT and SDT, while ZPD and Apprenticeship serve as sensitizing backdrops for interpreting patterns of scaffolding and fading. In our framework, AI support can reduce extraneous load; when learners still test, modify, and explain AI output, germane processing is retained (scaffolding). When delegation becomes rapid acceptance, germane processing may be suppressed (offloading). Motivationally, AI-enabled success can raise perceived competence, yet authorship can decline when evaluative control is ceded, producing a competence–autonomy tension whose direction depends on task design, timing, and norms. Table 1 links these gaps to our research questions. By anchoring analysis in CLT and SDT and using ZPD/Apprenticeship as sensitizing guides, our study moves beyond whether AI helps performance to examine how specific interaction patterns co-occur with cognitive load allocation and motivational appraisals.

Methods

Research design

This study employed a comparative, semester-long, multi-wave qualitative design grounded in Charmaz (2014)'s constructivist grounded theory (GT). Alternative designs (e.g., mixed-methods that integrate quantitative performance indicators with qualitative process-tracing) could address complementary questions. Our aim was inductive theory building, using GT to surface processes,

conditions, and perceived consequences (Glaser & Strauss, 2017).

The design has two key features: (a) it is semester-long (16 weeks) with repeated encounters that allow us to observe shifts in strategies and beliefs over time, and (b) it is comparative across two naturally occurring instructional ecologies. Specifically, we conduct constant comparison both within the AI-enabled section and across sections (AI-enabled vs. human pair programming) to surface patterns that appear unique to AI-mediated work. We treat the human pair-programming section as a theoretical contrast group, not a control, and we make no causal or efficacy claims. Section membership followed administrative enrollment in naturally occurring sections; sections shared the same syllabus and assignments but may differ in instructor, time slot, and peer norms. Inferences drawn from cross-section contrasts are therefore interpreted as suggestive for theory-building. This stance is consistent with Charmaz's use of contrasts to advance theoretical distinction in constructivist GT (Charmaz, 2014).

The pair-programming and AI-enabled sections are intentionally non-equivalent ecologies (human–human collaboration vs. AI-supported solo work). We use the contrast to sharpen categories via constant comparison and avoid generalizing beyond these ecologies. All cross-ecology observations are framed as *ecology-conditioned*, and we center inferences on within-ecology patterning (e.g., how strategies evolve over time inside each ecology).

We integrate data across sources (interviews, interaction logs, concept maps) using data-source triangulation and employ analytic triangulation (multiple analysts) to challenge emerging interpretations. We return to comparative contingencies and evidentiary limits in the Limitations section. Detailed instruments and rubrics are provided in the Supplement: concept mapping (Supplementary Material A), interview protocol and stimulus-recall procedures (Supplementary Material B), and prompt-type coding rubric (Supplementary Material C).

Setting and participants

Researcher description and positionality

The research team consists of educators and HCI researchers who have both used generative AI in their own work and observed its use by students. We approached this study with a dual perspective: we recognize AI's pedagogical potential and remain concerned about dependency that can undermine deep learning. This stance guided our inquiry toward documenting both "virtuous" and "vicious" cycles rather than seeking to confirm a preconceived view. To ensure the transparency of our position, we maintained reflexive memos throughout data collection and analysis and periodically reviewed coding decisions with attention to this dual perspective.

Setting, recruitment, and participant selection

The study was conducted during the Fall 2024 semester in a required, second-year undergraduate Java programming course at a public university in Shanxi province, China. Instruction and interviews were conducted in Mandarin; all sections followed the same syllabus, assignments, and assessment schedule, with section-level policy differences concerning AI use as described below. The course focused on object-oriented programming principles and data structures. All procedures were approved by the university's Institutional Review Board.

Participants were recruited via voluntary response to an in-class announcement and a follow-up screening survey. From the pool of volunteers, we used a multistage purposive sampling strategy (Patton, 2002) to select information-rich cases. Our goal was to recruit participants representing a broad range of prior experience, GPA, and motivation to facilitate a robust constant-comparison analysis. The final sample size (41 students) was determined by theoretical sufficiency rather than statistical power. We monitored the emergence of first-order codes across the semester and observed that late-semester interviews did not introduce new first-order categories; subsequent data served to densify and elaborate existing categories. We report stopping logic and the audit trail in the Trustworthiness and Rigor subsection. The sample comprised two groups:

- *The AI-User Group (N=24)*: Students from sections where the use of a university-provided generative AI tool (GitHub Copilot in IntelliJ) was permitted and encouraged.
- *The Theoretical Contrast Group (N=17)*: Students from a section emphasizing traditional human-human pair programming, used as a theoretical contrast to surface collaborative verification and discourse practices that may differ from solo AI use. We do not interpret this group as a control nor as the absence of AI.

Section assignment was administrative (i.e., not randomized). Potential differences in instructor, time slot, and peer norms are acknowledged; our comparative claims are therefore framed as suggestive contrasts that refine categories rather than as causal effects.

Participant flow and characteristics

All participants who were recruited completed the study. Baseline demographic data, collected via the screening survey, are summarized in Table 2. We report baseline

Table 2 Participant demographics and baseline experience by group

Characteristic	AI-user group (*N* = 24)	Theoretical contrast group (*N* = 17)
Average Age (years)	20.0 (*SD* = 1.0)	20.2 (*SD* = 1.2)
Gender (% Female)	29.2% (*n* = 7)	35.3% (*n* = 6)
Prior Java Experience	7.5 months (*SD* = 2.8)	7.1 months (*SD* = 3.3)
Self-Reported Skill	62.5% Beginner, 37.5% Intermediate	59% Beginner, 41% Intermediate
Mean GPA (*SD*)	3.40 (0.33)	3.37 (0.36)

descriptors to support transparency and transferability; given non-random section membership, these summaries are not used for statistical inference. Additional contextual descriptors collected to support thick description (e.g., prior pair-programming exposure and attitudes toward AI) are summarized in the Supplement.

Data collection procedures

To capture a behavioral trace, all IDE activity for the AI-User group was recorded using a custom, open-source IntelliJ logging plugin. To protect privacy, all data was anonymized at the source; student identifiers were replaced with randomly generated IDs before transmission to our secure research server. The raw logs were processed using a Python script that parsed the data to extract key features for each student-AI interaction: the timestamp, the full text of the user's prompt (defined below) and, where applicable, code context for accepted completions, the AI-generated suggestion, and a binary flag indicating whether the code was accepted or rejected. This pipeline produced structured data that supported descriptive summaries (e.g., prompt frequency, acceptance rates) that contextualize, but do not replace, the primary qualitative analysis.

We define a prompt as any user-initiated natural-language instruction to the assistant within the IDE chat panel, or an explicit acceptance of a generated completion that is preceded by a typed intent comment (e.g., "complete the file parser"). Pure auto-completions without explicit user instruction were excluded from classification analyses because they lack an intentional prompt to analyze.

We classified prompts into two categories using a coding rubric (examples in Supplementary Material C): (1) Transactional Generation (imperatives that request code/fixes with minimal rationale, e.g., "write a method to parse CSV and handle errors"), and (2) Conceptual Inquiry (questions that request explanations, trade-offs, or principles, e.g., "why prefer final here?" or "what are the implications of using streams for this loop?"). Two coders independently labeled a stratified subset of $N = 220$ prompts (approximately 26% of the corpus of 857 solution-seeking prompts), sampled across early/late semester windows and multiple assignment types within the AI-user group. Disagreements were resolved by consensus, and the refined rubric was then applied to the full corpus (see Supplementary Material C for coder training, sampling, and agreement details).

To reduce over-reliance on a two-bin scheme, we added a *complementary* "verification-intensity" rating (0–2) to the same stratified subset ($N = 220$): 0 = none/shallow (accept with no local checks), 1 = local test/edit (e.g., minimal run or small refactor before commit), 2 = test-first/refactor (e.g., write/run tests or structured revision before acceptance). This axis allowed us to distinguish Transactional prompts that nevertheless elicited meaningful verification.

Across AI users, we analyzed 857 solution-seeking prompts. Of these, 75% (N = 643) were Transactional Generation and 25% were Conceptual Inquiry. We report these counts descriptively to contextualize patterns.

Interaction occurred in IntelliJ IDEA (2024 release line) with GitHub Copilot enabled under the university license. The IntelliJ plugin captured timestamp, anonymized user ID, prompt text, action (accept/reject), selected code context for accepted completions, and an anonymized commit hash; identifiers were replaced at source. Prompts and commits were aligned to assignment due times to derive a days-to-deadline variable used descriptively.

Pre- and post-course concept mapping

To track changes in students' conceptual knowledge structures, participants from both groups completed a concept mapping task in the first and final weeks of the course. Students were given 15 key terms related to Object-Oriented Design (e.g., Inheritance, Polymorphism, Encapsulation) and asked to arrange them on a digital canvas and draw/label links to represent relationships (the detailed concept mapping task and rubric is provided in Supplementary Material A). Maps were scored independently by two raters using a rubric assessing (A) valid propositions (link accuracy and linking phrases), (B) hierarchical structure, and (C) cross-links. Inter-rater reliability was assessed using a two-way random-effects intraclass correlation, ICC(2,1), for the total score, and quadratic-weighted Cohen's κ for the ordinal subscales (Hierarchy and Cross-links); rubric details, edge-case rules, and reliability estimates appear in Supplementary Material A. We use concept maps as one converging indicator of integrative understanding; results are descriptive and intended to triangulate interviews and logs rather than adjudicate efficacy. Because concept maps preferentially capture hierarchical and cross-link structure, we treat them as a converging indicator only and avoid using them to infer metacognition without corroboration from interviews and logs.

Post-course semi-structured dyadic interviews

Interviews provided the primary qualitative material through 60-minute, semi-structured dyadic interviews conducted after final exams. The complete interview schedule, dyad composition rules, and stimulus-recall selection criteria (conceptual inquiry; delegation near deadline; large acceptance later linked to an error) are provided in Supplementary Material B. We employed a stimulus recall technique (Vaivio, 2012), showing AI-users anonymized snippets of their own interaction logs and code to prompt reflection on their in-the-moment thinking (e.g., "Can you walk me through why you accepted this suggestion but rejected the next one?"). Interviews were conducted in Mandarin, the native language of the participants, by a native-speaking member of the research team. Audio was transcribed and translated into English. To check fidelity, a second translator conducted a back-translation into Mandarin. Discrepancies were resolved by consensus among bilingual researchers familiar with programming pedagogy. We used dyads to leverage stimulated recall and to increase comfort for participants accustomed to pair work. To mitigate potential social desirability and peer influence, we paired participants within comfort/ability bands, reminded them that disagreement was welcome, and scheduled interviews after grading was complete. Translation was conducted by bilingual researchers familiar with programming pedagogy. Discrepancies between forward and back translations were resolved by consensus. Coding drew primarily on Mandarin transcripts; English was used for reporting quotations. To surface possible overshadowing, coders flagged deference cues (e.g., hedging, topic withdrawal) and recorded both dominant and deferred accounts as paired incident memos. We did not conduct individual follow-ups, which is a limitation; discrepant or silenced cases are retained as negative evidence when integrating categories.

Data analysis

We followed an iterative constructivist grounded theory process (Charmaz, 2014) supported by NVivo 14. Interview transcripts were the primary material; interaction logs and concept maps provided triangulation to contextualize and, when necessary, challenge emergent categories. We used constant comparison within the AI-enabled section and across the two ecologies to refine categories. We additionally employed analytic triangulation by involving multiple analysts in independent coding, memo exchange, and consensus meetings. Across all stages, we used constant comparison both within the AI-enabled section and across the two naturally occurring instructional ecologies to refine categories. The analysis proceeded in overlapping phases: initial coding, focused coding with constant comparison, and theoretical integration through memoing and diagramming.

Phase 1: Initial (Open) coding We began with line-by-line, gerund-focused coding of early interviews and salient log episodes (stimulated-recall anchors), staying close to participants' language (e.g., "seeking quick validation," "testing before trusting," "outsourcing boilerplate"). Memos captured nascent relationships (e.g., deadline pressure → delegation) and noted negative/edge cases for later audit.

Phase 2: Focused coding and constant comparison We elevated the most explanatory initial codes to focused codes and compared incident-to-incident within and across sections. Cross-ecology contrasts were used as theoretical contrasts, not controls, to sharpen distinctions around verification practices and discourse. Two discrepant-case routines were pivotal: (a) a participant's interview description of "working together with the AI" was contradicted by a log pattern of one-shot imperatives and immediate acceptance; this led us to label the category as Partnership Framing; (b) a case where a student's concept map showed dense cross-links despite extensive transactional prompting; this negative case tempered a simple offloading narrative and motivated a conditional link between "prompt craft" and reflective explanation.

Phase 3: Theoretical coding and model development We integrated categories via iterative memoing, diagramming, and return-to-data checks. Memo cadence was weekly during active collection and biweekly during consolidation. Codebooks were versioned at major checkpoints (early focus codes; cross-ecology contrast refinement; pre-model freeze). Iterative analytic memos documented how "deadline-driven delegation" and "verification stance" coalesced into the model's two feedback loops; these decisions are reflected in the finalized codebook (Table 3).

Intercoder agreement and consensus Two analysts independently coded a stratified subset of interviews (and associated log excerpts) covering both instructional ecologies and early/late semester timepoints. Agreement was assessed on a stratified subset. For prompt-type labels, coder training, sampling, and agreement statistics are reported in Supplementary Material C. For interview coding of higher-order categories, independent coding was followed by consensus meetings; disagreements were resolved and rules added to the versioned codebook. Two boundary rules illustrate how disagreements shaped the codebook: (1) *Explain-then-generate* prompts were coded as Inquiry only when the log showed an intervening local test or refactor *before* code acceptance; otherwise they remained Transactional. (2) *Partnership Framing* was split to include an Illusion-of-Dialogue subtheme after discrepant cases where partnership language co-occurred with one-shot acceptances in logs. These

Table 3 Codebook for student–AI interaction patterns in programming Education

Code	Code name	Definition and examples
S1: Partnership Framing & Discourse		
PART	Partnership Framing	Students discursively describe the AI as a collaborative partner or tutor. May legitimize shortcuts as "learning." E.g., "It felt like a senior programmer next to me."
ILLU	Illusion of Dialogue	Discursive–behavioral mismatch: students report "working together," but logs show one-shot imperatives and rapid acceptance with minimal negotiation
ANTH	Anthropomorphic Metaphors	Attributing human qualities/pedagogy to AI (e.g., "24/7 tutor," "it understands what I need")
S2: Strategy Repertoire & Switching		
SOCR	Socratic Inquiry	Prompts seeking explanation, alternatives, trade-offs, or tests; iterative small edits and local checks. E.g., "Why prefer final here?" "Show two designs with pros/cons."
TRAN	Transactional Delegation	Strategy-level delegation seeking complete code with minimal rationale (often aligns with TG prompts in Supplement C). E.g., "Write the entire GUI save-button listener with try/catch."
DELE	Deadline Delegation (amplifier)	Increased reliance on TRAN as deadlines approach (final 24–36 h); larger insertions and fewer checks
DANC	Strategic Dance	Dynamic switching between SOCR and TRAN as functions of time pressure, element interactivity/novelty, and familiarity
S3: Evaluation & Verification Practices		
TRUST	Trust-but-Can't-Verify	Novices accept unfamiliar constructs (e.g., streams) and "hope it runs," followed by symptom-level debugging when errors arise
BOIL	Boilerplate Blindspot	More experienced students offload "trivial" glue code to AI and accept uncritically; later defects originate in this boilerplate
UNCR	Uncritical Acceptance	Immediate integration without tests/modification; shallow or absent verification before committing
S4: Metacognition & Self-Appraisal		
TLSE	Tool-Linked Self-Efficacy	Confidence tethered to steering the assistant ("prompt craft" as primary skill)
CONF	Mastery Conflation	Conflating Tool Mastery with Domain Mastery (e.g., equating good prompts with programming competence)
AMC	Attenuated Metacognitive Calibration	Mismatch between perceived readiness and independent capability; uncertainty about AI-off performance
FRAG	Fragile Competence	Strong project performance coexisting with fear of unassisted exams; brittle understanding under AI-off conditions

rules were added to the versioned codebook and back-applied to earlier incidents. Throughout, we avoided causal language in category definitions and anchored all relations in observed patterns plus participants' articulated rationales.

Trustworthiness and Rigor

We addressed credibility, dependability, confirmability, and transferability following Lincoln and Guba (1988), tailoring procedures to constructivist GT and our two-ecology design. We used: (a) semester-long, multi-wave engagement (16 weeks); (b) triangulation across data sources (interviews, logs, concept maps) and analysts; and (c) member checking with six participants (three AI-enabled; three pair-programming), whose feedback refined labels and boundary statements. An audit trail of reflexive journals, dated analytic memos, and version-controlled codebooks (milestones: initial open codes; focused codes; cross-ecology refinements; pre-model freeze) is maintained. Instruments and procedures are documented in Supplementary Materials A–C.

We emphasize transferability via thick description of the institutional setting, section-level AI policies, language of instruction (Mandarin), tooling, and assignment ecology. Comparative claims are framed as suggestive contrasts given non-random section membership and possible instructor/time/peer-norm differences. We practiced reflexivity through team memos that explicitly tracked our dual stance (pedagogical promise vs. dependency risk). Periodic memo audits examined where this stance might bias code application (e.g., prematurely labeling prompts as "offloading") and prompted re-reading of counterevidence before codebook updates.

The appearance of new first-order codes was monitored separately within each ecology. When successive waves in an ecology produced no new first-order codes, subsequent interviews primarily densified existing categories (S1–S4). We ceased recruitment at 41 participants and continued analytic refinement. Stopping logic is summarized here; coder training and agreement for prompt labeling are detailed in Supplementary Material C (C3), stimulus-recall selection in Supplementary Material B, and concept-map scoring and reliability in Supplementary Material A.

Findings

Partnership framing

A frequently observed pattern in the AI-enabled section was a discursive tendency to describe the AI as a collaborative partner, even when the logged exchanges were largely transactional. We treat this as a discursive framing; in some cases it aligned with behavior, in others it did not.

When interviewed, students consistently framed their relationship with the AI in anthropomorphic terms, describing it as an active agent in their learning. This tendency was widespread. For example, Chen, a student with moderate prior experience, explained his process as a partnership:

> *"It felt like having a senior programmer sitting right next to me. When I got stuck, I would explain the problem to it, and we would work through the problem together. It's much faster than trying to find an answer on your own." (Chen, AI-User)*

Similarly, Aisha emphasized the AI's role as a pedagogical guide, highlighting a sense of collaborative dialogue:

> *"I treat it like a 24/7 tutor. I can ask it anything, and it helps me think about the code from a different angle. The back-and-forth really helps solidify what I'm supposed to be doing." (Aisha, AI-User)*

However, a systematic analysis of the interaction logs often showed one-shot imperative prompts followed by rapid acceptance, a pattern more consistent with transactional delegation than with negotiated co-construction. Figure 1 schematizes a representative sequence from Chen's logs that exemplifies this discursive–behavioral mismatch.

In such cases, the partnership framing helps make sense of a powerful shortcut as pedagogically legitimate. Importantly, not all cases fit the mismatch pattern. In a negative case, Hui (AI-User) described "going back and forth until it made sense," and the logs aligned: Hui prompted for an explanation, requested a smaller function variant, ran a local test, and then asked for a refactor to improve readability before accepting.

> *"I asked it to explain the stream first, then I tried it and it failed on empty input, so I told it to handle that and show me the steps. After that I tweaked the names and it looked right." (Hui, AI-User)*

Here, the partnership framing and behavior cohere: inquiry, revision, and testing were observable.

The strategic dance: from socratic inquiry to deadline-driven delegation

Students did not adopt a single, static strategy for using AI. A commonly observed pattern we term the Strategic Dance involves dynamic shifts between Socratic Inquiry and Transactional Delegation as conditions change (e.g., deadline proximity, task complexity, and familiarity). Descriptively, early assignment periods were characterized by exploratory questions and small, iterative commits; approaching deadlines, many students pivoted to directive prompts and larger code insertions. In addition, unfamiliar or high–element-interactivity tasks prompted returns to inquiry even late in a cycle, whereas familiar subroutines (e.g., file I/O, simple parsing) were more often delegated quickly.

The first mode, Socratic Inquiry, was characterized by exploratory, conceptual questions aimed at understanding principles. This learning-oriented strategy was most common early in an assignment cycle when time was abundant. The second and more dominant mode, Transactional Delegation, involved issuing direct commands to generate functional code. This performance-oriented strategy became more prevalent as assignment deadlines loomed and when tasks were perceived as routine. Figure 2 illustrates this pattern using excerpts from one representative student, Li, contrasting his interaction styles at two different points in a project timeline.

Li's own reflection during his interview consciously articulated this widespread pragmatic shift, highlighting the rational calculation behind it:

> *"At the start, I try to use it to understand things. I have time, so I ask it to explain concepts I'm shaky on. But when it's 2 AM and the project is due at 9 AM... I'm not trying to learn anymore, I'm just trying to finish. I just ask it for the code I need to make it work. It's about survival at that point." (Li, AI-User)*

To add temporal granularity, we reconstructed per-student timelines by aligning prompts and commits to assignment due dates and tagging tasks by perceived complexity/familiarity (from interview recall). Descriptively, the proportion of Transactional Delegation prompts increased within the final 24–36 h before deadlines, while spikes in Socratic Inquiry also occurred when students encountered novel APIs or multi-component interactions (e.g., GUI event handling + file persistence). Overall, in our data the Strategic Dance reflects adaptive strategy switching sensitive to time pressure and task affordances. In the Discussion we relate these descriptive patterns to CLT (extraneous vs. germane processing) and SDT (perceived competence vs. authorship/autonomy).

write a java method to sort an arraylist of Customer objects by last name

Here's a simple Java method to sort an ArrayList of Customer objects by their last name:

```
public class Main {
    public static void main(String[] args) {
        ArrayList<Customer> list = new ArrayList<>();
        list.sort((a, b) -> a.getLastName().compareTo(b.getLastName()));
        for (Customer c : list) {
            System.out.println(c);
        }
    }
}
```

Accept Reject

User accepts the entire 8-line code block without modification.

now write the part that reads customer data from "customers.csv" and puts it into the arraylist

Sure! Here's a complete and Java code that for IO.

```
try (BufferedReader reader = new BufferedReader(new FileReader("cust
            String line;
            while ((line = reader.readLine()) != null) {
                String[] parts = line.trim().split(",");
                if (parts.length == 2) {
                    String firstName = parts[0].trim();
                    String lastName = parts[1].trim();
                    customers.add(new Customer(firstName, lastName));
                } else {
                    System.out.println("Invalid line: " + line);
                }
            }
        }
}
```

Accept Reject

User accepts the entire 12-line code block without modifcation.

Fig. 1 Analytic schematic of a representative interaction sequence illustrating the Illusion of Dialogue sub-theme. The student describes collaboration, yet the logs show one-shot imperatives and immediate acceptance of generated code (left) with minimal negotiation or modification

The double-edged sword of critical evaluation

Beyond interaction strategies, many AI-section participants described difficulty evaluating AI outputs. Students recognized that suggestions could be flawed or suboptimal, yet their verification practices were uneven. We present two recurrent–but not universal–patterns observed in interviews and aligned logs.

The Novice's Dilemma: trust but can't verify

For less experienced students, the challenge manifested as a pattern we term the Novice's Dilemma. This state of anxious reliance was reported by many self-identified beginners. These students possessed enough knowledge to understand the AI was fallible but lacked the deeper expertise to confidently assess its correctness or efficiency. Wang, a student in this category, vividly described this predicament:

> *"I know Copilot can be wrong... but half the time, the code it gives me is more complex than what I would write, so I don't even know how to check if it's right. It uses some weird Java stream thing, and I just have to cross my fingers and run it. It's stressful because you don't know if you're learning the right way or just making a bigger mess." (Wang, AI-User)*

This anxiety was borne out in the interaction logs of several novice participants. A common pattern involved

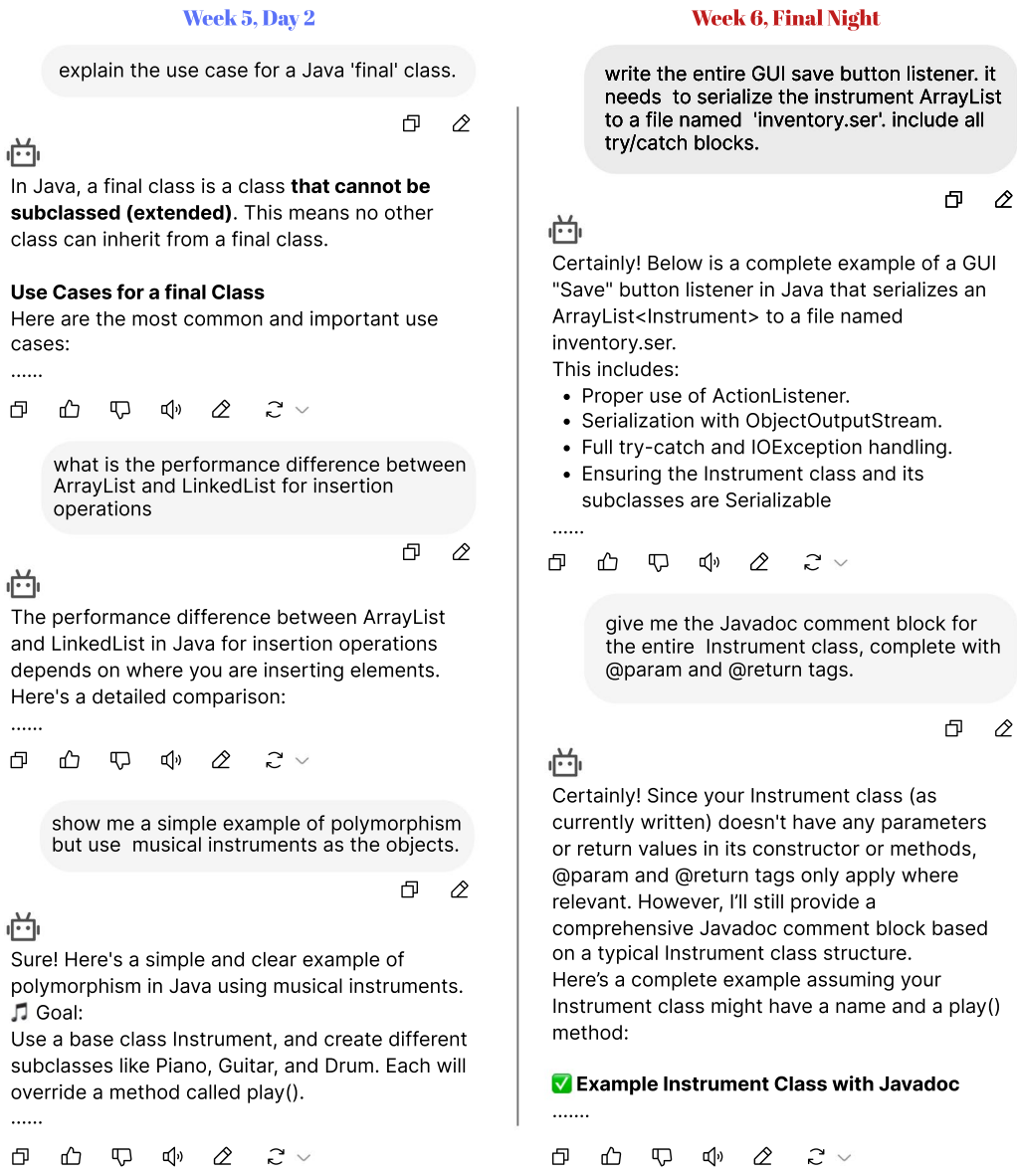

Fig. 2 Visualization of the Strategic Dance. Left: early-cycle log segments showing Socratic Inquiry (e.g., "explain...", "what is the difference...") with small, iterative commits. Right: late-cycle segments dominated by Transactional Delegation (e.g., "write the entire... listener,"“give me the Javadoc...") with larger insertions. This figure is illustrative rather than inferential

accepting a multi-line, syntactically elegant code block from the AI that used an advanced API unfamiliar to the student. When this code contained a subtle logical error, the student was unable to identify the root cause. Their commit histories would reveal long, frustrating debugging sessions, with commit messages like "another fix" or "why is it not working," where they attempted to fix symptoms in other parts of their code.

This experience contrasted with accounts from novices in the human-pair theoretical contrast group. When faced with code they didn't understand, they engaged in direct clarification-seeking with their partner. As one theoretical contrast group novice explained, "I just kept asking my partner, 'Can you explain that line again?' until it finally clicked." This contrast is consistent with social verification opportunities in human pairing.

The boilerplate blindspot

More experienced students were not immune to evaluation lapses; a different pattern emerged that we label the Boilerplate Blindspot. Confident in their core algorithmic skills, these students often offloaded tasks they perceived as "trivial" or "boilerplate" to the AI, reserving their own cognitive effort for the "hard parts." This created a critical blind spot. Mei, one of the top-performing students, recounted her experience:

> *"It was so frustrating. I spent two hours debugging my complex scheduling algorithm, convinced the logic was wrong somewhere. But the error was actually in the simple CSV parsing code that Copilot wrote at the beginning. I didn't even look at it because, I mean, it's just reading a file. I just assumed it would be correct." (Mei, AI-User)*

Mei's interaction log and Git history confirm this account as illustrative of this pattern. Early in a session, she would issue a prompt for boilerplate code (e.g., '// Java code to parse data from data.csv') and immediately accept the generated block. Her subsequent commits would show intense focus on her complex algorithm. The final commit, hours later, would reveal the error was in the "simple" AI-generated code she had uncritically accepted.

As a theoretical contrast, the collaborative nature of their work created a natural "quick cross-check." As one experienced student in the theoretical contrast group noted,

> *"Even on simple stuff, my partner might look over my shoulder and say, 'Wait, shouldn't that be...?' We catch each other's dumb mistakes."*

This distributed process of peer review, even on trivial code, was described as a safeguard against the Boilerplate Blindspot. We treat these differences as contextual contrasts that refine categories.

Metacognitive outcomes: attenuated calibration and tool-linked self-efficacy

Across the semester, interview accounts and artifacts suggested shifts in how students monitored and appraised their learning. We describe two related outcomes: an emphasis on tool-linked self-efficacy and signs of attenuated metacognitive calibration. We present these as co-occurring patterns within this sample rather than causal effects. Many AI-section students described growing pride in "prompt craft" and in steering the tool effectively. Some articulated this skill as if synonymous with programming competence, suggesting a potential conflation of Tool Mastery with Domain Mastery. Below we show both aligning and misaligning cases. Because interviews occurred near high-stakes exams, elevated anxiety is a plausible alternative account; we therefore treat these as signals consistent with attenuated calibration.

When prompt craft helps–and when it doesn't

A recurring theme in the interviews with more than half of the AI-user group (14 of 24) was a developing pride in their "prompt craft." For these students, the primary skill they felt they were honing was not algorithmic thinking but the ability to write a perfect prompt to elicit a desired output from the AI. Their focus shifted from the cognitive work of decomposing a problem to the tactical work of reverse-engineering a problem description into an effective query. Zhang, a student who received high marks on all his AI-assisted projects, articulated this shift clearly:

> *"In the beginning, I was just asking simple questions. By the end, I was a pro. I learned that getting the right answer is all about asking the right question, with the right context and examples. My real skill now is writing a perfect prompt that gets me the code I need in one shot." (Zhang, AI-User)*

This sentiment was common. Students described a sense of accomplishment derived from their successful manipulation of the AI. While this is undeniably a valuable new skill, their language revealed a possible conflation: they often spoke of this "prompt engineering" skill as if it were synonymous with programming skill itself. Their self-efficacy became tethered to their ability to manage the tool, rather than their ability to reason from first principles. In a contrasting case, Peng reported using the assistant to surface alternatives first–"give me two designs with pros/cons and a failing test for each"–then wrote and ran the tests before accepting any code. Peng described the prompts as scaffolds for reasoning rather than substitutions for it, and his commit history showed test-first edits before code acceptance. This case illustrates how prompt craft can also support reflective practice.

In our material, prompt craft contributed positively when it entailed (a) decomposition into constraints and interfaces, (b) contrastive requests (two designs with pros/cons), (c) test or failure-case generation prior to code acceptance, and (d) traceability requirements (e.g., "explain invariants you rely on"). These practices align with our Scaffolding Loop. Risks arose when prompt craft was optimized for one-shot code acquisition without verification, aligning with Offloading.

Indicators of attenuated metacognitive calibration

We observed indicators consistent with attenuated metacognitive calibration–mismatch between perceived readiness and independent capability–co-occurring with frequent delegation patterns for some students. We triangulate interview statements with concept-map artifacts, treating the latter as one descriptive lens.

The most salient evidence emerged from the comparative analysis of the pre- and post-semester concept maps. While both groups demonstrated learning, the quality of that growth differed descriptively. Figure 3 visually contrasts the average gains in concept map complexity, with the human-pair theoretical contrast group showing larger increases in cross-links and hierarchical organization.

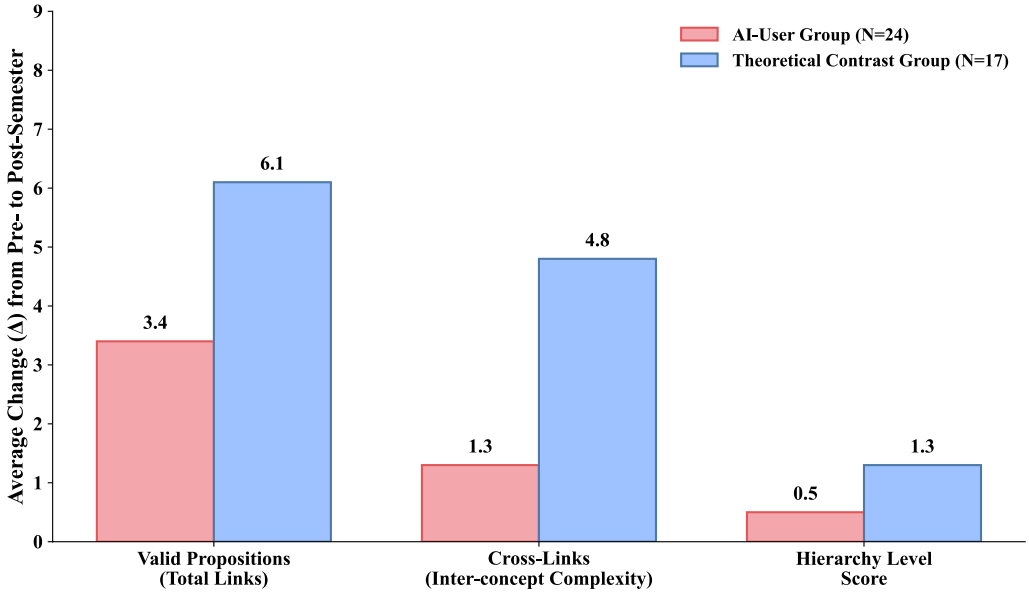

Fig. 3 Descriptive summary of pre- to post-semester changes in concept-map structure. Bars are means only; no statistical tests were conducted. Values are used to contextualize the qualitative themes

We interpret these summaries cautiously as converging, descriptive evidence rather than inferential tests.

This summary was mirrored in the students' own subjective self-assessments. The AI-users often expressed a vague sense of unease–a dissonance between their high project grades and their perceived knowledge. This was perfectly encapsulated by Liu in the final days of the course:

> *"I got good grades on all the projects, but now I have the final exam next week without Copilot... and honestly, I'm terrified. I'm not sure what I actually know." (Liu, AI-User)*

Liu's fear contrasts with the self-assessments of the human-pair participants. Having engaged in constant dialogue, debate, and argumentation, they were more articulate and accurate about their knowledge gaps. As one theoretical contrast participant, David, noted, "My partner and I argued for an hour about interfaces, so I know for a fact I don't really get them yet." These accounts suggest that collaborative discourse provided a metacognitive mirror for some students.

Taken together, these patterns–tool-linked efficacy narratives, artifacts suggesting different degrees of integrative structure, and expressed uncertainty–cohere with the notion of attenuated metacognitive calibration for some AI-section participants. In the Discussion, we relate these observations to CLT (opportunities for germane processing) and SDT (perceived competence vs. authorship/autonomy), and we outline design moves aimed at strengthening evaluation and reflective practice.

Discussion

A model of domain versus tool mastery

The core contribution is a grounded theory tension model that describes how learners navigate between Domain Mastery (transferable concepts, evaluation, and explanation) and Tool Mastery (procedural efficiency with AI). Rather than a strict dichotomy, our data suggest co-development pathways shaped by boundary conditions such as time pressure, task complexity/element interactivity, prior familiarity, and scaffolding design/norms. We characterize two recurrent loops: a Scaffolding Loop and an Offloading Loop. Learners can move between loops over time, and both patterns may co-exist within the same assignment. We describe associations rather than causal mechanisms. This model helps reconcile the "performance–learning paradox" noted in recent empirical and review work (Yan et al., 2025; Denny et al., 2024). Large-sample and field studies report robust performance and affect benefits with GenAI (e.g., efficiency, accuracy, reduced anxiety) (Fan et al., 2025; Yilmaz & Yilmaz, 2023; Lyu et al., 2024; Shihab et al., 2025), and meta-analyses further suggest positive effects on both lower- and higher-order outcomes as well as motivation/engagement, moderated by discipline, learning strategy, and context (Xia et al., 2025; Qi et al., 2025). Our Scaffolding/Offloading loops offer a process-level account of how such gains can arise under different conditions: inquiry-oriented uses align with deeper processing, while rapid delegation supports speed and affect but may compress opportunities for durable learning.

Scaffolding loop When students have time or adopt a mastery orientation, they often begin with Socratic

Inquiry and pursue modification, testing, and explanation of AI suggestions. In CLT terms, this pattern is consistent with reduced extraneous effort (less syntax/search churn) while retaining or increasing germane processing (reasoning about structure and tests) (Debue & Van De Leemput, 2014; Gkintoni et al., 2025). It also coheres with the literature on "productive struggle" and productive failure: structured difficulty and delayed direct instruction can strengthen long-term knowledge organization when learners engage in explanation and error analysis. In SDT terms, students described competence signals arising from their evaluation and authorship (autonomy) (Niemiec & Ryan, 2009; Xia et al., 2022). Over time, this loop appeared to co-occur with more selective delegation and more critical use of the AI.

Offloading loop Under deadline pressure or on familiar/routine tasks, many students pivoted to Transactional Delegation and sometimes accepted outputs with shallow testing. In CLT terms, this pattern is consistent with reduced engagement in germane processes when acceptance is rapid, especially on high element-interactivity problems where understanding relations among components is crucial (Stadler et al., 2024). It also resonates with the expertise-reversal effect: supports that aid novices can be redundant for more advanced learners or can displace self-regulated processing (Kalyuga, 2007). In SDT terms, students reported immediate competence signals (task completion) even as evaluative control and authorship (autonomy) could recede (Niemiec & Ryan, 2009; Xia et al., 2022). This offers a process-level account of how performance and engagement metrics can improve (Fan et al., 2025; Yilmaz & Yilmaz, 2023; Lyu et al., 2024; Qi et al., 2025) while markers of conceptual integration or transfer may lag (Jose et al., 2025; Bastani et al., 2024). The Boilerplate Blindspot observed here also aligns with industry reports that routine scaffolding from AI can speed delivery on "brownfield" tasks while concealing subtle faults in glue code (Shihab et al., 2025). For a subset of participants, we describe attenuated metacognitive calibration –a mismatch between perceived readiness and independent capability–consistent with reduced opportunities for explanation and test construction. Figure 4 presents the tension model with the two loops and boundary conditions.

Crosswalk to CLT and SDT

To situate the model within established frameworks, we provide a compact crosswalk indicating how the observed categories align with CLT and SDT constructs. These mappings are interpretive anchors rather than tests of theory; they are consistent with prior accounts of load management (Debue & Van De Leemput, 2014; Gkintoni et al., 2025; Stadler et al., 2024) and motivational needs in learning environments (Niemiec & Ryan, 2009; Xia et al., 2022).

In sum, the Scaffolding Loop aligns with CLT patterns of extraneous reduction with germane engagement and with SDT patterns supporting autonomy and calibrated competence; the Offloading Loop aligns with reduced germane engagement and with a competence–autonomy

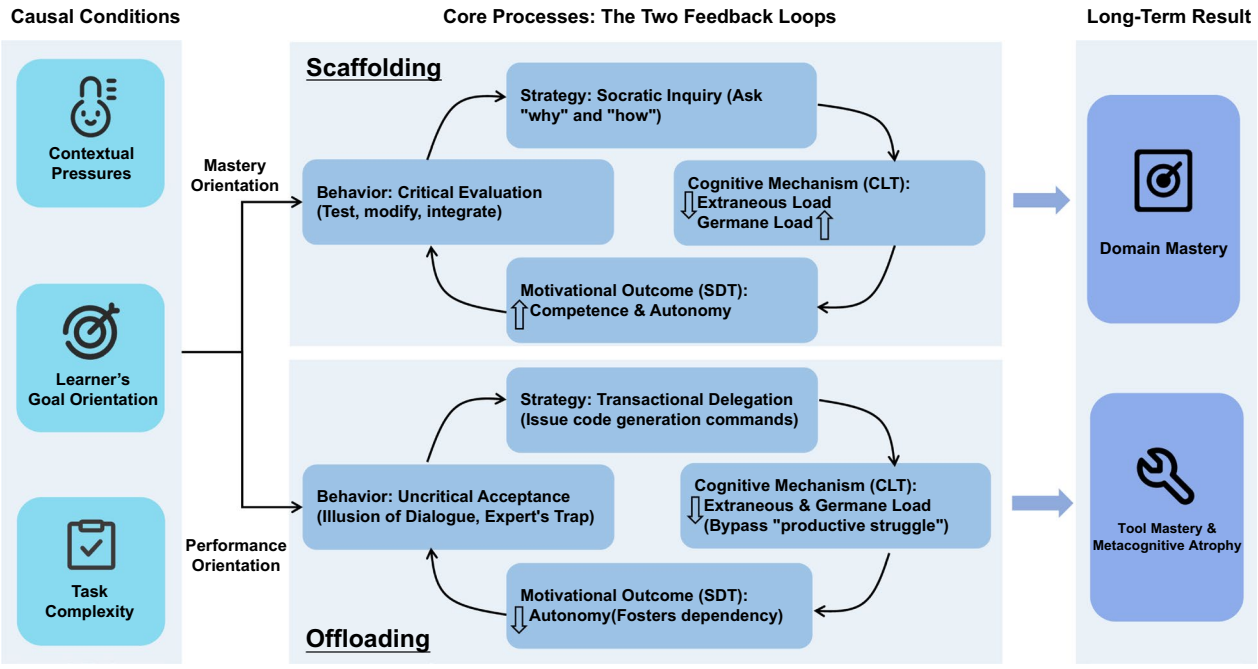

Fig. 4 A model of Domain vs. Tool Mastery with two recurrent loops (Scaffolding and Offloading) and boundary conditions (time pressure, task complexity/familiarity, scaffolding design, norms). The model summarizes patterns observed in this study and is offered as a theory-building synthesis

Table 4 Interpretive crosswalk from observed categories to CLT and SDT constructs (descriptive, non-causal)

Observed category	CLT-oriented interpretation	SDT-oriented interpretation
Socratic Inquiry	Often lowers extraneous effort (less search/syntax churn) while retaining/increasing germane processing (modify/test/explain)	Supports authorship/autonomy and calibrated competence via self-evaluation
Transactional Delegation	When acceptance is rapid, germane engagement may be suppressed; routine tasks delegated quickly; risks amplified under high element interactivity	Raises perceived competence (completion) but may reduce perceived authorship/autonomy if evaluative control is ceded
Partnership Framing (Illusion of Dialogue subtheme)	Processing depth is ambiguous; dialogue language does not guarantee germane engagement	Can boost perceived competence/relatedness even when control over reasoning is limited
Boilerplate Blindspot	Uncritical acceptance of "simple" code can increase extraneous debugging and mask sources of error	Confidence in "hard parts" may coexist with lower authorship over routine components
Attenuated Metacognitive Calibration	Fewer opportunities for germane explanation/testing align with weaker calibration signals in artifacts	Competence signals are tool-mediated; authorship cues are thinner, complicating self-assessment

tension wherein perceived effectiveness rises as authorship/control may decline. These patterns offer a mechanism-level interpretation of the mixed but generally positive outcome literature (Fan et al., 2025; Yilmaz & Yilmaz, 2023; Lyu et al., 2024; Shihab et al., 2025; Xia et al., 2025; Qi et al., 2025) alongside warnings about shallow conceptual change (Jose et al., 2025; Bastani et al., 2024).

Synthesis in relation to the research questions

We summarize our answers to the research questions using descriptive, non-causal language, integrating interviews, logs, and artifacts and drawing on the CLT/SDT crosswalk (Table 4).

RQ1 (Strategies)

Students did not exhibit a single stable approach; instead, we observed a Strategic Dance between Socratic Inquiry and Transactional Delegation (Fig. 2). Shifts co-occurred with boundary conditions: (a) deadline proximity (delegation typically increased within 24–36 h of due times), (b) task complexity/element interactivity (novel APIs and multi-component interactions elicited renewed inquiry), and (c) familiarity (perceived "routine" sub-tasks were more often delegated quickly). This temporal/granularity account complements single-session experimental findings of performance benefits (Yilmaz & Yilmaz, 2023) by indicating when and how students pivot modes over authentic timelines. Interpretively, inquiry phases tend to reduce extraneous effort while retaining germane processing, whereas rapid delegation can suppress germane engagement when acceptance/testing is shallow (Debue & Van De Leemput, 2014; Stadler et al., 2024).

RQ2 (Evaluation)

Evaluation practices were uneven. Two recurrent patterns were observed. For novices, a Trust-but-Can't-Verify dilemma appeared when students recognized fallibility yet lacked the evaluative repertoire to assess correctness or efficiency; logs showed acceptance of unfamiliar constructs followed by prolonged symptom-level debugging, echoing concerns about shallow understanding with GenAI (Jose et al., 2025). For more experienced students, a Boilerplate Blindspot emerged when routine components were offloaded and accepted uncritically, later becoming the source of hard-to-locate faults; this nuance helps interpret why reported productivity gains on maintenance tasks (Shihab et al., 2025) can co-exist with latent defects in glue code. As a theoretical contrast, the pair-programming section described social verification (rapid questioning and shoulder-surf review) that sometimes surfaced trivial errors early. In CLT terms, both patterns can elevate extraneous debugging or limit germane engagement; in SDT terms, immediate competence cues from progress may coexist with thinner authorship signals when evaluative control is ceded (Niemiec & Ryan, 2009).

RQ3 (Metacognition & Self-Efficacy)

Across the semester, some AI-section participants narrated tool-linked self-efficacy–confidence tied to steering the assistant–alongside signs of attenuated metacognitive calibration, expressed as uncertainty about unassisted performance despite strong project records. Concept-map summaries provide a converging, descriptive lens in which the pair-programming section showed larger increases in cross-links and hierarchical organization; we interpret these artifacts cautiously and do not claim efficacy differences. Notably, we also documented contrasting cases where "prompt craft" functioned as a scaffold for reflection (e.g., soliciting alternatives and failure cases, writing tests before acceptance), indicating that

appropriate task design can channel tool use toward explanation and calibration. In the CLT/SDT crosswalk, these outcomes are consistent with fewer opportunities for germane explanation/testing and with a competence–autonomy tension shaped by task design, timing, and norms (Debue & Van De Leemput, 2014; Niemiec & Ryan, 2009).

Alternative explanations and rival hypotheses

Several alternative explanations merit consideration alongside this model. First, observed contrasts between the AI-enabled section and the human pair-programming section may reflect the social verification affordances of collaboration rather than effects attributable to AI per se. Participants in the theoretical contrast group frequently described side-by-side explanation, immediate challenge, and shared error-finding–practices that can scaffold evaluation regardless of tool use. Our analysis therefore interprets these contrasts as contextual refinements of categories, not as indicators of differential efficacy. Second, concept mapping offers only one window on conceptual structure and may preferentially capture hierarchical relations over other legitimate forms of knowledge organization common in programming (e.g., component–interface patterns or test-driven schemas). Descriptive differences in maps should thus be read as one converging lens, not as a definitive proxy for competence or transfer. Third, students' partnership language can be metaphorical. The Partnership Framing category includes cases where discourse and behavior align (back-and-forth revision and testing), alongside Illusion of Dialogue cases where they diverge; we avoid inferring deception or naiveté from metaphorical talk. Finally, reports of uncertainty prior to unassisted exams may partly reflect typical pre-assessment anxiety. We triangulate such reports with logs and artifacts, but we cannot fully disentangle anxiety from calibration within this design.

Practical implications

The model proposes several instructional design strategies that, if tested, could strengthen evaluation and reflective practice while preserving the benefits of AI support. Assignments could incorporate an explicit critique-the-AI phase in which students must locate, test, and justify modifications to assistant output; such a phase targets germane processing (CLT) and authorship/autonomy (SDT) and aligns with calls to go beyond surface metrics toward process-sensitive pedagogy (Denny et al., 2024; Jeyaraj & Sethi, 2025). Instructors can pilot planned fading by designating sub-tasks that must be attempted AI-off (e.g., writing tests, tracing invariants) before any tool assistance is permitted, thereby rehearsing independent evaluation and addressing the competence–autonomy tension (Niemiec & Ryan, 2009). Short verification journals or commit-message templates that capture how outputs were tested and why changes were made may foster a durable evaluation script without imposing excessive overhead. Contrastive prompting–requesting multiple solution variants, anticipated failure cases, or trade-offs–can turn delegation into a scaffold for explanation. Instructors can make this ambidexterity explicit by grading for contrastive prompts, test-first requests, and traceable rationale, thereby steering prompt craft toward explanation rather than acquisition. Finally, periodic AI-off checkpoints aligned with exam conditions can help calibrate perceived readiness against independent capability. We present these as hypothesis-generating implications: they are grounded in the observed loops and our CLT/SDT crosswalk but require controlled study to establish effects and boundary conditions. Concrete exemplars already exist: error-code–driven reflective reports that make students explain and repair their own faults (Lee et al., 2025), and PA-GPT cycles that simulate peer review with ChatGPT as a structured "virtual peer" (Wu et al., 2025b). Both map onto our Scaffolding Loop by converting AI from a code generator into a trigger for explanation, testing, and critique. Importantly, these gains appear contingent on the structure of interaction–reflection prompts or peer-cycle requirements–rather than tool presence alone.

Limitations and future work

This study's interpretations are bounded by several limitations. Our interpretations are bounded by the ecology-specific design: sections differed in instructor/time/peer norms and were *not* constructed for controlled contrasts. We therefore emphasize *transferability* via thick description rather than statistical generalization. A single 16-week semester, while adequate for constant comparison across waves, does not match the multi-year span common in some grounded-theory traditions; thus, our categories should be read as theoretically sufficient for this context rather than maximally elaborated. The cultural and linguistic context (Mandarin instruction at a single Chinese public university) enhances ecological validity for that setting but affects transferability beyond this setting; cross-cultural replications are needed to examine how authority relations, technology attitudes, and collaboration norms shape the observed loops. Measurement scope is also limited: concept maps provide a narrow indicator of integrative structure and may not align with all forms of programming expertise; future work should incorporate transfer tasks, delayed retention assessments, authentic maintenance challenges, and code-review quality metrics. Dyadic interviews, while effective for stimulated recall, can invite peer influence and social desirability; individual follow-ups and

think-alouds could mitigate these pressures. We adopt theoretical sufficiency rather than universal saturation; additional sampling across institutions and curricula may surface further variants of the Strategic Dance, Partnership Framing, and Boilerplate Blindspot. Our interpretation of calibration signals is constrained by timing: interviews near examinations confound anxiety with metacognitive accuracy.

Future studies can strengthen inference through solo-AI versus solo-non-AI comparisons, within-course crossover designs, or randomized scaffolding variants (e.g., mandated critique phases or fading protocols). Richer instrumentation–fine-grained logs of testing actions, edits preceding acceptance, and automated traces of explanation requests–would enable closer linkage between evaluation practices and outcomes. Incorporating metacognitive measures such as calibrated confidence judgments and judgment-of-learning prompts, alongside SDT-aligned surveys of autonomy, competence, and relatedness, would clarify the competence–autonomy tension under different designs (Niemiec & Ryan, 2009; Xia et al., 2022). Finally, multi-site replications and preregistered analyses can assess the transportability of the tension model and delineate the conditions under which AI support functions as a scaffold versus a shortcut (Xia et al., 2025; Qi et al., 2025; Shihab et al., 2025).

Conclusion

This study offers a process-level, theory-building account of AI-assisted learning in an authentic undergraduate programming context. Anchored in Cognitive Load Theory and Self-Determination Theory, the proposed tension model describes how learners navigate between Domain Mastery (concepts, explanation, evaluation) and Tool Mastery (procedural efficiency with AI), and how boundary conditions (time pressure, task complexity/familiarity, and scaffolding design) may be associated with movement between a Scaffolding Loop and an Offloading Loop. Rather than adjudicating efficacy, our contribution helps explain how widely reported performance and affect gains can co-occur with thinner opportunities for germane processing and authorship, and why some students exhibit attenuated metacognitive calibration when offloading becomes routine. Practically, the model introduces specific strategies designed to leverage AI's benefits while strengthening the user's critical evaluation and reflection. These methods include dedicated 'critique-the-AI' phases, planned fading of AI assistance through offline tasks, verification journals, and contrastive prompting. Conceptually, this approach shifts the conversation from a simple 'use or ban' debate to a more nuanced discussion of how and when the tool aligns with learning goals and motivational needs.

We invite future work to test this model's boundary conditions via multi-site replications, randomized scaffolding variants, and richer instrumentation of evaluation practices (e.g., test-first traces, edits before acceptance) as well as calibrated metacognitive measures. In this way, the present account functions as a roadmap for empirical verification: it specifies the processes to observe, the tensions to manage, and the instructional levers to test as AI becomes a durable feature of programming education.

Supplementary Information

The online version contains supplementary material available at https://doi.org/10.1186/s40594-025-00592-w.

Supplementary file 1

Acknowledgements

We are grateful to the anonymous reviewers for their constructive comments and valuable suggestions, which significantly improved the quality and clarity of this manuscript.

Author contributions

DL: Conceptualization, Methodology, Software, Investigation, Data Collection, Writing — Original Draft. GF: Formal Analysis, Validation, Supervision, Project Administration, Writing — Review & Editing. LP: Resources, Funding acquisition, Writing — Review & Editing. All authors have read and approved the final manuscript.

Funding

This work was supported by the Natural Science Foundation of Shanxi province (No.202203021221145)
Shanxi Province Intelligent Software and Human Machine Environment System Postgraduate Joint Training Demonstration Base (No.2022JD11).

Data availability

The datasets used and/or analysed during the current study are available from the corresponding author on reasonable request.

Declarations

Ethics approval and consent to participate

All procedures involving human participants in this study were conducted in accordance with the ethical standards of the institutional research committee. The study protocol was reviewed and approved by Taiyuan University of Science and Technology's Institutional Review Board (Protocol No. 20240421). Informed consent was obtained from all individual participants included in the study prior to their participation.

Competing interests

The authors declare no Conflict of interest.

Author details

[1]Taiyuan University of Science and Technology, Taiyuan, China
[2]University of Malaya, Kuala Lumpur, Malaysia

Received: 1 July 2025 / Accepted: 28 December 2025
Published online: 13 March 2026

References

Bastani, H., Bastani, O., Sungu, A., Ge, H., Kabakcı, O., & Mariman, R. (2024). Generative AI can harm learning. SSRN Working Paper No. 4895486.

Boguslawski, S., Deer, R., & Dawson, M. G. (2025). Programming education and learner motivation in the age of generative AI: Student and educator perspectives. *Information and Learning Sciences,126*(1/2), 91–109.

Charmaz, K. (2014). Constructing grounded theory (2nd Ed). SAGE.
Chiu, T. K., Moorhouse, B. L., Chai, C. S., & Ismailov, M. (2024). Teacher support and student motivation to learn with artificial intelligence (AI) based chatbot. *Interactive Learning Environments,32*(7), 3240–3256.
Chiu, T. K. F., & Li, Y. (2023). How can emerging technologies impact STEM education? *Journal for STEM Education Research,6*(3), 375–384.
Chiu, T. K. F., Xia, Q., Zhou, X., Chai, C. S., & Cheng, M. (2023). Systematic literature review on opportunities, challenges, and future research recommendations of artificial intelligence in education. *Computers and Education: Artificial Intelligence,4*, Article 100118.
Collins, A., Brown, J. S., Holum, A., et al. (1991). Cognitive apprenticeship: Making thinking visible. *American Educator,15*(3), 6–11.
Debue, N., & Van De Leemput, C. (2014). What does germane load mean? An empirical contribution to the cognitive load theory. *Frontiers in Psychology,5*, 1099.
Denny, P., Prather, J., Becker, B. A., Finnie-Ansley, J., Hellas, A., Leinonen, J., & Sarsa, S. (2024). Computing education in the era of generative AI. *Communications of the ACM,67*(2), 56–67.
Dung, N.T.M. , Hieu, P.N.V. , Linh, N.T. , Han, N.T.N. , Trong, H. , & et al. (2024). The impact of students' using generative AI for learning on self-learning motivation: A study based on self-determination theory. The 2nd international conference: Resiliency by technology and design (rtd 2024) (pp. 362–382).
Fan, G., Liu, D., Zhang, R., & Pan, L. (2025). The impact of AI-assisted pair programming on student motivation, programming anxiety, collaborative learning, and programming performance: A comparative study with traditional pair programming and individual approaches. *International Journal of STEM Education,12*(1), 16.
Gkintoni, E., Antonopoulou, H., Sortwell, A., & Halkiopoulos, C. (2025). Challenging cognitive load theory: The role of educational neuroscience and artificial intelligence in redefining learning efficacy. *Brain Sciences,15*(2), 203.
Glaser, B., & Strauss, A. (2017). Discovery of grounded theory: Strategies for qualitative research. Routledge.
Gupta, S., & Bostrom, R.P. (2009). Technology-mediated learning: A comprehensive theoretical model. Journal of the Association for Information Systems 109686–714.
Jeyaraj, A., & Sethi, V. (2025). Generative ai tools in software teaching and learning: Implications for business education. *Journal of Management and Business Education,8*(1), 127–146.
Jose, B., Cherian, J., Verghis, A. M., Varghise, S. M., & Joseph, S. (2025). The cognitive paradox of AI in education: Between enhancement and erosion. *Frontiers in Psychology,16*, 1550621.
Kalyuga, S. (2007). Expertise reversal effect and its implications for learner-tailored instruction. *Educational Psychology Review,19*(4), 509–539.
Kapur, M. (2008). Productive failure. *Cognition and Instruction,26*(3), 379–424.
Khlaif, Z.N., Hamamra, B., & Hussein, E.T. (2025). AI paradox in higher education: Understanding over-reliance, its impact, and sustainable integration. Education and Information Technologies. (Advance online publication)
Lee, H. Y., Chen, P. H., Lin, C. J., Huang, Y. M., & Wu, T. T. (2025). Leveraging chatgpt for personalized reflective learning in programming education: effects on self-efficacy, higher-order thinking, and project implementation skills. Education and Information Technologies 1–40.
Leinwand, S., Brahier, D. J., Huinker, D., Berry, R. Q., Dillon, F. L., Larson, M. R., & Smith, M. S. (2014). *Principles to actions: Ensuring mathematical success for all*. National Council of Teachers of Mathematics: NCTM.
Létourneau, A., Deslandes Martineau, M., Charland, P., Karran, J. A., Boasen, J., & Léger, P. M. (2025). A systematic review of AI-driven intelligent tutoring systems (ITS) in K-12 education. *npj Science of Learning,10*(1), 1–13.
Li, Y. (2025). Advancing STEM education as a dynamic and distinct academic field. *International Journal of STEM Education,12*(1), 1.
Lincoln, Y. S., & Guba, E. G. (1988). *Criteria for assessing naturalistic inquiries as reports*. New Orleans, LA: Annual meeting of the american educational research association.
Lyu, W., Wang, Y., Chung, T., Sun, Y., & Zhang, Y. (2024). Evaluating the effectiveness of llms in introductory computer science education: A semester-long field study. In Proceedings of the eleventh ACM conference on learning@ scale (pp. 63–74).
Medika Life (n.d.). Llms as a modern partner in vygotsky's zone of proximal development. https://medika.life/llms-as-a-modern-partner-in-vygotskys-zone-of-proximal-development/34
Niemiec, C. P., & Ryan, R. M. (2009). Autonomy, competence, and relatedness in the classroom: Applying self-determination theory to educational practice. *Theory and Research in Education,7*(2), 133–144.
Patton, M. Q. (2002). *Qualitative research and evaluation methods (vol 3)*. Sage.
Qi, X., Weijia, L., Yiming, Y., Xiaojing, W., & et al . (2025). A systematic review and meta-analysis of the effectiveness of generative artificial intelligence (genai) on students' motivation and engagement. Computers and Education: Artificial Intelligence 100455.
Shabani, K., Khatib, M., & Ebadi, S. (2010). Vygotsky's zone of proximal development: Instructional implications and teachers' professional development. *English Language Teaching,3*(4), 237–248.
Shah, A., & Soosai Raj, A.G. (2024). A review of cognitive apprenticeship methods in computing education research. Proceedings of the 55th acm technical symposium on computer science education v. 1 (pp. 1202–1208).
Shihab, M.I.H., Hundhausen, C., Tariq, A., Haque, S., Qiao, Y., & Mulanda, B.W. (2025). The effects of github copilot on computing students' programming effectiveness, efficiency, and processes in brownfield coding tasks. 407–420.
Stadler, M., Bannert, M., & Sailer, M. (2024). Cognitive ease at a cost: LLMs reduce mental effort but compromise depth in student scientific inquiry. *Computers in Human Behavior,160*, Article 108386.
Stojanov, A. (2023). Learning with ChatGPT 3.5 as a more knowledgeable other: An autoethnographic study. *International Journal of Educational Technology in Higher Education,20*(1), 35.
Vaivio, J. (2012). *Interviews-learning the craft of qualitative research interviewing*. Taylor & Francis.
Vygotsky, L. S. (1978). *Mind in society: The development of higher psychological processes (vol 86)*. Harvard University Press.
Wu, F., Dang, Y., & Li, M. (2025). A systematic review of responses, attitudes, and utilization behaviors on generative ai for teaching and learning in higher education. *Behavioral Sciences,15*(4), 467.
Wu, T. T., Lee, H. Y., Chen, P. H., Lin, C. J., & Huang, Y. M. (2025). Integrating peer assessment cycle into Chatgpt for stem education: A randomised controlled trial on knowledge, skills, and attitudes enhancement. *Journal of Computer Assisted Learning,41*(1), Article e13085.
Xia, Q., Chiu, T. K. F., Lee, M., Sanusi, I. T., Dai, Y., & Chai, C. S. (2022). A self-determination theory (SDT) design approach for inclusive and diverse artificial intelligence (AI) education. *Computers & Education,189*, Article 104582.
Xia, Q., Zhang, P., Huang, W., & Chiu, T. K. F. (2025). The impact of generative AI on university students' learning outcomes via Bloom's taxonomy: A meta-analysis and pattern mining approach. *Asia Pacific Journal of Education,45*(1), 1–31.
Yan, L., Greiff, S., Lodge, J. M., & Gašević, D. (2025). Distinguishing performance gains from learning when using generative AI. *Nature Reviews Psychology,4*, 378–379.
Yilmaz, R., & Yilmaz, F. G. K. (2023). The effect of generative artificial intelligence (AI)-based tool use on students' computational thinking skills, programming self-efficacy and motivation. *Computers and Education: Artificial Intelligence,4*, Article 100147.

Publisher's Note

Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.